CROISSATTN: A COMPLETE FP4 ATTENTION BACKWARD

Anonymous authors
Paper under double-blind review

ABSTRACT

FP4 training speeds up the linear layers of language models, but at long context the cost shifts to attention. Four of attention's six products sit in the backward pass but existing FP4 recipes keep all or part of it in higher precision. CroissAttn extends native FP4 execution to both forward and all four backward attention products. It combines stochastic rounding of gradients and saved-factor recasts with a consistent softmax correction, preserving the quantized forward's straight-through gradient in conditional expectation. An error decomposition separates noise propagated through the backward pass from new product noise. Query-row-owned accumulation and packed storage reduce the memory required by tiled execution. On long-context workloads, native B200 attention kernels achieve 1.5–2.2× backward speedups over BF16. Training from scratch reaches 10B tokens with validation gaps of 0.0439 nats at 500M parameters and 0.0451 at 1.7B. Native Qwen3-8B continued training reaches a 0.056-nat gap. Under FP4 serving, its checkpoint achieves lower validation loss than directly cast BF16-trained weights. These results connect complete FP4 attention backward to training quality, deployment, and long-context execution.

1 INTRODUCTION

FP4 training reduces the cost of the linear layers in a language model. Attention becomes an increasing share of the remaining work as context grows. Projection work scales linearly with sequence length; attention compares pairs of tokens and scales quadratically. Extending FP4 execution to attention is therefore central to efficient long-context training. We evaluate training quality at 2,048-token contexts and measure execution speed and memory separately at longer contexts.

Attention contains two forward matrix products and four backward products. Existing methods cover different subsets of these operations. Projection-only recipes retain higher-precision attention, and quantized-attention recipes use a higher-precision backward or selected higher-precision products (Panferov et al., 2026; Zhang et al., 2026b; Ding et al., 2026). Figure 1 summarizes these precision choices. CroissAttn supplies FP4 operands to all six principal attention products.

Two challenges determine the design. First, the backward products are dependent. Rounding the probability gradient changes the score gradient, which then enters the query and key products. The saved forward factors also appear with new contraction axes and new quantization blocks. A complete backward must preserve one gradient target through these operations. Second, a low-precision product can leave large higher-precision buffers in memory. The execution schedule must reduce storage as well as operand precision.

CroissAttn addresses the first challenge by specifying the straight-through gradient of its quantized forward. Score replay uses the original forward codes. Each backward product rounds its gradient operand and recasts its saved factor with independent stochastic rounding. An auxiliary softmax correction preserves the target mean through the complete pass. The error model then separates saved-factor recast noise, propagated noise, and fresh rounding at the final products.

The execution schedule addresses the second challenge through query-row ownership. Each query tile accumulates its complete gradient locally and writes it once. Packed forward factors reduce storage, and tiled replay avoids materializing the attention matrix. The same description accounts for row normalization, contraction-axis layouts, and the higher-precision auxiliary correction.

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

We evaluate the method through matched precision ablations, training from initialization, native 8B continued training, and optimized long-context systems. Fixing the FP4 forward gives validation losses of 2.4914 with a higher-precision backward and 2.4913 with the FP4 backward. At 10B tokens, completing attention adds 0.024–0.025 nats beyond smoothed FP4 projections. At 8B, FP4-trained and BF16-trained checkpoints score 74.9% and 75.0% under BF16 serving. FP4 training improves validation under FP4 serving, where the model encounters the precision used during training.

Contributions. CroissAttn provides a complete FP4 attention backward with a specified conditional-mean target, an error decomposition for its dependent products, and a tiled implementation with query-row-owned accumulation. The evaluation connects these components to incremental quality cost, FP4 deployment, and long-context memory and throughput.

2 Related Work

Low-precision training. Mixed-precision and FP8 training combine narrow operands with higher-precision accumulation (Micikevicius et al., 2018; 2022). Sun et al. (2020) develop four-bit training with gradient quantization and range control. Wang et al. (2025) use a differentiable quantization estimator and outlier clamping with compensation for FP4 language-model training. Chmiel et al. (2025) study block size, scale format, and rounding, using nearest rounding in forward and stochastic rounding in backward and weight-gradient products. Quartet and NVIDIA's NVFP4 recipe develop native FP4 projection training (Castro et al., 2026; NVIDIA, 2025); Quartet II introduces MS-EDEN for microscaled gradient estimation (Panferov et al., 2026). These works establish the numerical and hardware foundations for FP4 training.

Low-precision attention. SageAttention2 combines outlier smoothing with low-bit attention products (Zhang et al., 2025). SageAttention3 introduces microscaled FP4 inference attention and studies an eight-bit training backward that retains the probability-gradient product in higher precision (Zhang et al., 2026a). Attn-QAT trains a quantized attention forward using a higher-precision backward (Zhang et al., 2026b). Full-Stack FP4 assigns FP4 to the score, query-gradient, and key-gradient products and BF16 to the remaining three principal attention products (Ding et al., 2026). CroissAttn completes the six-product FP4 path and analyzes its conditional mean and propagated error.

		BF16	Quartet II	Full-Stack FP4	CroissAttn
Projection	XW (fwd)	BF16	FP4	FP4	FP4
	$dX,\ dW$	BF16	FP4	FP4	FP4
Attention forward	$Q_f K_f^\top$	BF16	BF16	FP4	FP4
	$P_f V_f$	BF16	BF16	BF16	FP4
Attention backward	$dP = dO\, V_f^\top$	BF16	BF16	BF16	FP4
	$dV = P_f^\top\, dO$	BF16	BF16	BF16	FP4
	$dQ = \widehat{dS}\, K_f$	BF16	BF16	FP4	FP4
	$dK = \widehat{dS}^\top Q_f$	BF16	BF16	FP4	FP4
	softmax, accum.	FP32	FP32	FP32	FP32

Figure 1: **Matrix-product precision coverage.** CroissAttn uses FP4 operands in both forward and all four backward attention products. Softmax, accumulation, optimizer arithmetic, and the auxiliary row correction use higher precision. Projection-only controls in our experiments use our rounding recipe at the coverage shown for Quartet II.

Tiled execution and outlier control. FlashAttention reduces attention memory traffic by tiling and recomputing intermediates; later versions refine work partitioning and hardware pipelines (Dao et al., 2022; Dao, 2024; Shah et al., 2024; Zadouri et al., 2026). CroissAttn follows this tiled organization

and assigns query-gradient accumulation to one owner. Large activation channels motivate outlier handling (Dettmers et al., 2022). Our smoothed projection recipe uses the product-preserving rescaling of SmoothQuant (Xiao et al., 2023), with forward and backward detailed in Appendix C.5.

Stochastic rounding. Stochastic rounding preserves a covered scalar in expectation (Croci et al., 2022). Our analysis carries this property through layout-dependent recasts, softmax correction, and the final gradient products. It also tracks the covariance introduced when a rounded operand is reused within a product.

3 Attention Model

3.1 Operations and representation

For one causal attention head, $Q, K, V \in \mathbb{R}^{T\times d}$ contain queries, keys, and values. Query i uses keys $j \le i$. The usual forward is $P = \mathrm{softmax}(QK^\top/\sqrt{d})$ and $O = PV$. Given an incoming loss gradient dO, backward forms $dP = dOV^\top$, $dV = P^\top dO$, the softmax derivative, and the two products for dQ and dK.

NVFP4 stores E2M1 payloads with one E4M3 scale per 16 values and an FP32 multiplier (NVIDIA, 2025; 2026). A decoded value includes all of its scales. We write a subscript f for a decoded forward factor, a star for the target gradient, and a hat for a stochastic approximation. The FP4 matrix instructions consume the packed representations. Softmax, row reductions, and accumulation retain higher precision.

3.2 Quantized forward and gradient target

The forward computes scores $s = Q_f K_f^\top/\sqrt{d}$ and fixes the final row maximum m_i. It then rounds the exponentials before normalization:

$$U_{ij} = \exp(s_{ij} - m_i), \qquad \ell_i = \sum_j U_{ij}, \qquad L = \mathrm{diag}(\ell),$$
$$U_f = \mathcal{Q}(U), \qquad P = L^{-1}U, \qquad P_f = L^{-1}U_f, \qquad O = P_f V_f. \tag{1}$$

Forward FP4 quantization rounds Q_f, K_f, V_f, U_f to nearest; masked entries are zero. Permitted exponentials have unit row maxima, anchoring their forward scale; the denominator uses unrounded exponentials. U_f is the packed operand and P_f the effective normalized coefficient matrix.

We differentiate through the surrogate $P_{\mathrm{STE}} = P + \mathrm{sg}(P_f - P)$, where sg preserves its argument's value and stops its derivative. This surrogate has forward value P_f and softmax Jacobian J_P. With identity straight-through derivatives for the saved factors and fixed quantizer scales, the target is

$$dP_\star = dOV_f^\top, \qquad dV_\star = P_f^\top dO,$$
$$dS_\star = J_P(dP_\star)/\sqrt{d}, \qquad dQ_\star = dS_\star K_f, \qquad dK_\star = dS_\star^\top Q_f, \tag{2}$$

where $J_P(X) = P \odot [X - \mathrm{rowsum}(P \odot X)]$, broadcasting row sums across keys. Our dS includes $1/\sqrt{d}$. The value gradient uses the forward coefficients P_f; the score derivative uses the softmax probabilities P.

A two-key example. Consider $p = (3/4, 1/4)^\top$ and probability gradient $u = (2, -2)^\top$. The row correction is $D_\star = p^\top u = 1$, and the logit gradient is $p \odot (u - D_\star \mathbf{1}) = (3/4, -3/4)^\top$. For a rounded gradient $\widehat{u} = u + \xi$, auxiliary correction gives this target plus $\mathrm{diag}(p)\xi$. A centered ξ therefore preserves the target mean. This is the local calculation used in the complete backward.

4 CroissAttn Design

4.1 Complete stochastic backward

Stochastic rounding (SR) chooses adjacent represented levels $a < b$ with

$$\Pr(\widehat{x} = b) = \frac{x-a}{b-a}, \qquad \mathbb{E}\widehat{x} = x, \qquad \mathrm{Var}(\widehat{x}) = (x-a)(b-x), \quad x \in [a, b]. \tag{3}$$

A covering scale places every input inside the represented range. The backward below uses *two-operand SR*: independent stochastic rounding of gradient operands and saved-factor recasts.

The implemented backward uses the high-precision auxiliary-output construction of Attn-QAT (Zhang et al., 2026b, Section 2.3):

$$O' = PV_f, \qquad D_{\star,i} = \langle dO_i, O'_i \rangle = \sum_j P_{ij}(dP_\star)_{ij}. \tag{4}$$

The auxiliary product and reduction use higher precision. Let $\mathcal{M}(A, B)$ denote the product of independently SR-quantized FP4 operands. The complete backward is

$$\begin{aligned} \widehat{dP} &= \mathcal{M}(dO, V_f^\top), & \widehat{dV} &= \mathcal{M}(U_f^\top, L^{-1}dO), \\ \widehat{dS} &= P \odot (\widehat{dP} - D_\star)/\sqrt{d}, & \widehat{dQ} &= \mathcal{M}(\widehat{dS}, K_f), & \widehat{dK} &= \mathcal{M}(\widehat{dS}^\top, Q_f). \end{aligned} \tag{5}$$

Each product recasts its saved factor into the required contraction-axis layout. All FP4 training backward products use two-operand SR, including from-scratch and continued-training runs. E1 uses the same policy. The training and systems experiments use auxiliary correction. Streamed sampled centering needs an extra row-reduction pass and replay of the same draws (Appendix C.1).

Theorem 1 (Conditional mean)**.** *Condition on the saved forward values and incoming dO, denoted $\mathcal{F}_0$. Assume exact score replay and auxiliary correction, covering scales fixed before payload draws, independent operand streams within each product, and fresh conditionally unbiased draws at subsequent products. Then the represented backward in Equation* (5) *satisfies*

$$\mathbb{E}[\widehat{dV} \mid \mathcal{F}_0] = dV_\star, \qquad \mathbb{E}[\widehat{dQ} \mid \mathcal{F}_0] = dQ_\star, \qquad \mathbb{E}[\widehat{dK} \mid \mathcal{F}_0] = dK_\star. \tag{6}$$

Proof. Independent SR gives $\mathbb{E}[\mathcal{M}(A, B) \mid A, B] = AB$. The correction is affine in $\widehat{dP}$ with all other terms fixed, so $\mathbb{E}[\widehat{dS} \mid \mathcal{F}_0] = dS_\star$. Conditioning on $\widehat{dS}$ and applying the tower property gives the query and key means. The value mean follows directly from its single product. □

4.2 Replay, recasts, and normalization

Score replay. Backward recomputes the probabilities from the forward-layout packed queries and keys. It reuses their saved payloads and scales, with the same masks, block membership, and reduction order. A two-pass forward supplies a common final row maximum. Score replay performs no fresh query/key quantization.

Backward layouts. NVFP4 blocks run along the contraction axis. Changing that axis changes the values grouped under each scale. The saved factors are therefore decoded and re-quantized for their backward consumers (Table 1). Their SR draws use fresh, independent streams. Each recast contributes its own variance to the backward error model.

Table 1: **Saved-factor layouts.** Score replay uses the original query/key codes. Each principal backward product recasts its saved factor.

Factor	Forward contraction	Backward product	New contraction
Q_f	Channels	$\widehat{dS}^\top Q_f$	Queries
K_f	Channels	$\widehat{dS} K_f$	Keys
V_f	Keys	$dO V_f^\top$	Channels
U_f	Keys	$U_f^\top (L^{-1} dO)$	Queries

Value-gradient normalization. The query index is the contraction axis of dV. Its row-dependent normalizer belongs inside the product:

$$dV_\star = (L^{-1}U_f)^\top dO = U_f^\top (L^{-1} dO). \tag{7}$$

The gradient rows are scaled by $1/\ell_i$ before rounding. Two-operand SR preserves the means of the scaled gradient and the numerator recast. This factorization implements the forward normalization at every contracted query.

4.3 QUERY-ROW-OWNED EXECUTION

In a query-row-owned (QRO) pass, each query tile's owner visits permitted key tiles, accumulates the full query gradient locally, and writes it once. Separate key/value passes reduce contributions to shared heads. Probabilities and score gradients are generated per tile. This eliminates the global partial-query-gradient buffer and avoids storing a $T \times T$ attention matrix.

Algorithm 1 Query-row-owned FP4 backward pass

Require: Forward factors and replay metadata; incoming dO; auxiliary correction $D_\star$.

```
for each query tile I, assigned to one owner do
    Initialize higher-precision accumulator G_I ← 0.
    for each permitted key tile J do
        Replay P_IJ from the original score-layout codes.
        dP̂_IJ ← M(dO_I, V_{f,J}^⊤).
        dŜ_IJ ← P_IJ ⊙ (dP̂_IJ − D_{⋆,I})/√d.
        G_I ← G_I + M(dŜ_IJ, K_{f,J}).
    end for
    Write dQ̂_I ← G_I.
end for
```

Storage. FP4 payloads and their microscale bytes require $1/2 + 1/16 = 0.5625$ bytes per value, plus tensor multipliers. The packed-storage variant also retains working-weight and non-attention factors without auxiliary BF16 copies. The auxiliary output O' is retained or recomputed to form $D_\star$; its product is included in step timing. Saved-factor storage, auxiliary-output lifetime, and transient accumulators are separate parts of the memory budget.

Streamed scales. For a finite gradient tile X, let $A_X = \max|X|$. A sufficient covering construction selects an FP32 multiplier $g_X > 0$ and positive E4M3 block scales b_B satisfying

$$g_X \geq \frac{A_X}{6 \cdot 448}, \qquad b_B = \mathrm{ceil}_{\mathrm{E4M3},+}\left(\frac{\max_{x \in B}|x|}{6 g_X}\right). \tag{8}$$

The decoded scales are checked before payload draws. Each tile's multiplier is restored to its partial product before accumulation. This local rule applies to the realized gradient tile, including randomness from earlier products. Appendix B specifies range handling and replay.

4.4 ERROR PROPAGATED THROUGH THE BACKWARD

For a product AB, write $\widehat{A} = \bar{A} + \varepsilon_A$ and $\widehat{B} = \bar{B} + \varepsilon_B$, with centered errors and entrywise variances $\mathsf{V}_A = \mathbb{E}\varepsilon_A^{\odot 2}$ and $\mathsf{V}_B = \mathbb{E}\varepsilon_B^{\odot 2}$. Independent payload errors give

$$\mathbb{E}(\widehat{A}\widehat{B} - AB)^{\odot 2} = (\bar{A}\bar{B} - AB)^{\odot 2} + \mathsf{V}_A \bar{B}^{\odot 2} + \bar{A}^{\odot 2}\mathsf{V}_B + \mathsf{V}_A \mathsf{V}_B. \tag{9}$$

Powers are entrywise; adjoining matrices are multiplied. The four terms are squared bias, first-operand noise, second-operand noise, and their interaction. Two-operand SR retains the saved-factor variance and interaction. Deterministic recasts under gradient-only SR enter through the bias term, derived in Appendix A.5.

Let $\widehat{dP} = dP_\star + \Xi$ and define $\mathcal{A}(\Xi) = P \odot \Xi/\sqrt{d}$ for auxiliary correction. Write $r_Q(S) = \mathbb{E}[\|\mathcal{M}(S, K_f) - SK_f\|_F^2 \mid \mathcal{F}_0, S]$.

Theorem 2 (Backward error decomposition)**.** *Under Theorem 1 and finite second moments,*

$$\mathbb{E}\|\widehat{dQ} - dQ_\star\|_F^2 = \underbrace{\mathbb{E}\|\mathcal{A}(\Xi)K_f\|_F^2}_{\textit{propagated error}} + \underbrace{\mathbb{E} r_Q(\widehat{dS})}_{\textit{fresh product error}}. \tag{10}$$

The key-gradient identity uses $\mathcal{A}(\Xi)^\top Q_f$ and the risk of $\mathcal{M}(S^\top, Q_f)$. The value-gradient error is the risk of its single product.

Proof. The propagated error is fixed given $\widehat{dS}$. Fresh rounding has zero conditional mean, so its expected cross term with the propagated error is zero. Expanding the squared norm gives the result.□

This decomposition identifies error already present before the final query/key products. Its covariance depends on which rounded operands are reused across keys. The final-product risk is evaluated on the sampled $\widehat{dS}$, including its selected scales. Appendix A gives the full covariance calculation. Appendix C.1 compares auxiliary and sampled row corrections on fixed tensors.

5 Evaluation

5.1 Experimental setup

Models and training. Training from initialization covers 181M, 500M, and 1.7B parameters. Continued training uses Qwen3-1.7B and Qwen3-8B on FineWeb-Edu (Qwen Team, 2025a;b; Penedo et al., 2024). Quality experiments use 2,048-token contexts. The 500M/1.7B scratch studies and all Qwen3-8B FP4 runs execute native FP4 matrix instructions. Both single-GPU 8B arms store Adam first and second moments in BF16. GPU resource constraints limited replication to three paired seeds for the 181M/100M-token study and one seed per arm for the other training experiments. Appendix D gives the training and evaluation settings.

Precision controls. The controls are BF16, FP4 projections with BF16 attention, FP4 attention with BF16 projections, and a higher-precision STE backward on the same FP4 forward. Smoothed arms use $\alpha = 0.3$. All FP4 training backward products use two-operand SR and auxiliary correction. Projection-only controls use our rounding and smoothing recipe; their name specifies product coverage. Loss is reported in nats. Task scores are percentages, and the three-seed training statistic is mean $\pm$ sample standard deviation.

Systems. Optimized-kernel benchmarks use eight B200 GPUs with 167.6 GiB per GPU, tensor parallelism eight, sequence parallelism, and micro-batch two. Both precisions recompute SwiGLU intermediates. The eight-RTX PRO 6000 Blackwell study uses 95 GiB per GPU, tensor parallelism eight, micro-batch one, BF16 residuals, and no activation checkpointing. Memory and throughput are reported together in Section 5.4.

5.2 Training quality with FP4 attention

FP4 backward adds no visible penalty. To isolate the effect of backward precision, Table 2 compares two attention-only arms with the same FP4 attention forward and BF16 projections. After 200M continued-training tokens, validation loss is 2.4914 with higher-precision STE backward and 2.4913 with FP4 stochastic backward, a difference of only 0.0001 nats. Both arms remain about 0.064 nats above the BF16 baseline. Thus, the gap to BF16 is already present with higher-precision backward; replacing it with FP4 stochastic backward adds no visible loss penalty in this single-seed, matched-forward comparison.

Table 2: **Precision and smoothing ablations at 1.7B.** Seed 42, 200M continued-training tokens, 762 steps. The attention-only arms share the same FP4 forward. HP denotes higher-precision STE backward; F/B denotes forward/backward operand precision. Smoothed arms use $\alpha = 0.3$.

Arm	Projection	Attention F/B	Smooth	Validation	Gap
BF16	BF16	BF16/BF16	–	2.4278	0
Attention only	BF16	FP4/HP	–	2.4914	+0.064
Attention only	BF16	FP4/FP4	–	2.4913	+0.064
Projection only	FP4	BF16/BF16	No	2.6609	+0.233
Projection only	FP4	BF16/BF16	Yes	2.4856	+0.058
CroissAttn	FP4	FP4/FP4	No	2.7828	+0.355
CroissAttn	FP4	FP4/FP4	Yes	2.5562	+0.128

Smoothing substantially reduces the gap. Smoothing reduces the gap to BF16 from 0.355 to 0.128 nats for CroissAttn and from 0.233 to 0.058 nats for FP4 projections alone. With smoothed FP4 projections fixed, switching attention from BF16 to FP4 adds 0.071 nats.

The gap remains stable late in training. The 600M-token continuation ends at validation losses of 2.3980 for BF16 and 2.5114 for CroissAttn, a gap of 0.1134 nats. The gap remains between 0.111 and 0.114 nats over the final 700 steps.

Similar validation gaps at 10B tokens. When trained from initialization, the 500M and 1.7B models reach 10B tokens with validation gaps of 0.0439 and 0.0451 nats relative to BF16 (Table 3). Their smoothed FP4 projection-only controls each have a 0.020-nat gap. Switching attention from BF16 to FP4 therefore adds about 0.024–0.025 nats at these endpoints. At 500M, the full FP4 recipe's gap increases from 0.0380 at 1B tokens to 0.0439 at 10B tokens.

Table 3: **Training from initialization.** Loss gaps relative to BF16, in nats. All validation arms use projection smoothing ($\alpha = 0.3$). Training-loss results average over windows; the three-seed result reports the mean $\pm$ sample standard deviation of paired gaps. The 511M-token attention-only comparison uses QK normalization.

Parameters	Tokens	FP4 coverage	Gap to BF16	Seeds
Validation loss				
500M	1B	Projections + attention	0.0380	1
500M	10B	Projections only	0.020	1
500M	10B	Projections + attention	0.0439	1
1.7B	10B	Projections only	0.020	1
1.7B	10B	Projections + attention	0.0451	1
Training loss (window averages)				
181M	100M	Projections + attention	0.0315 ± 0.0011	3
181M	511M	Attention only	0.042	1

Small-model checks. At 181M parameters and 100M tokens, FP4 projections and attention yield a window-averaged training-loss gap of 0.0315 ± 0.0011 nats across three paired seeds. A separate attention-only CroissAttn ablation, with QK normalization and matched seed and data order, ends at 511M tokens with a final 200-step mean gap of 0.042 nats (Figure 2). Appendix D.2 details the seed decomposition and aggregation.

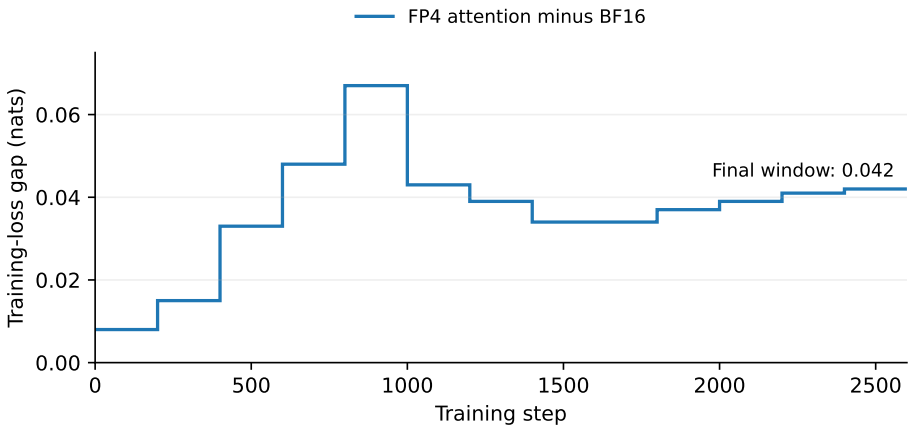


Figure 2: **Attention-only ablation of CroissAttn at 181M parameters.** FP4-attention minus BF16-attention training loss over 2,600 steps (511M tokens). Each segment is a 200-step mean. Both arms use QK normalization and the same seed and data order. The final-window gap is 0.042 nats.

5.3 NATIVE 8B TRAINING AND FP4 DEPLOYMENT

A 0.056-nat validation gap at 8B. After 200M continued-training tokens, Qwen3-8B reaches validation losses of 2.1839 with CroissAttn (native FP4 with smoothing) and 2.1278 with BF16. The gap stays between 0.054 and 0.059 nats across recorded evaluations from step 400 onward. Under BF16 serving, seven-task averages are 74.9% for CroissAttn and 75.0% for BF16 training; CroissAttn scores 73.3% under FP4 serving.

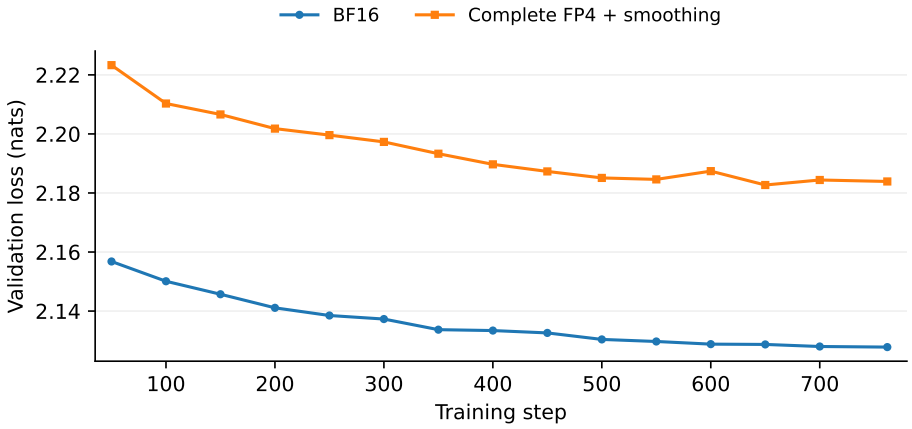


Figure 3: **Native Qwen3-8B continued training.** Recorded validation evaluations through 762 steps, one seed per arm. Both arms use 2,048-token training contexts. The final reported gap is 0.056 nats.

FP4 training improves FP4 serving. In a separate fixed-batch checkpoint evaluation (Table 4), CroissAttn reduces FP4-serving loss relative to directly cast BF16-trained weights by 0.0198 nats at 8B and 0.1060 nats at 1.7B. Switching the same checkpoints from BF16 to FP4 serving adds 0.0348 nats after CroissAttn training versus 0.0768 after BF16 training at 8B; the corresponding penalties at 1.7B are 0.0544 and 0.2214 nats. Direct casting uses no evaluation-time smoothing. Appendix D.3 gives the per-task results and checkpoint evaluation protocol.

Table 4: **Checkpoint loss by training and serving precision.** Fixed-batch validation loss in nats; bold marks the lower loss in each column. The 8B and 1.7B checkpoints follow 200M and 600M continued-training tokens, respectively, with one seed per arm. CroissAttn uses FP4 training with smoothing. Direct casting serves BF16-trained weights in FP4.

	Qwen3-8B		Qwen3-1.7B	
Training	BF16 serving	FP4 serving	BF16 serving	FP4 serving
BF16	**2.1282**	2.2050	**2.3983**	2.6197
CroissAttn	2.1504	**2.1852**	2.4593	**2.5137**

5.4 OPTIMIZED LONG-CONTEXT SYSTEMS

Kernel and step throughput. Native B200 attention kernels achieve paired speedups of 1.4–1.7× in forward and 1.5–2.2× in backward (Table 5). The complete Qwen3-8B system reaches 168k tokens/s at 32,768 tokens and 80k at 131,072 tokens, compared with 105k and 53k for BF16. Complete-step timing includes projection products, attention, packing, auxiliary correction, optimizer work, and exposed communication.

Storage and recomputation. At 524,288 tokens, the reported QRO configuration uses 151.9 GiB per GPU; adding packed working-weight and non-attention storage lowers the total to 93.6 GiB. Full-checkpoint BF16 uses 95.2 GiB. CroissAttn reaches this capacity using SwiGLU recomputation and runs at 26k tokens/s, compared with 14k for full-checkpoint BF16. Appendix E separates parameter state, saved activations, and temporary buffers.

Table 5: **Native B200 attention kernels.** Causal attention, head dimension 128, long-sequence workloads. Rates are effective PFLOP/s. Speedups are paired workload ratios.

Pass	BF16	FP4	Paired speedup
Forward	1.4–1.6	2.0–2.6	1.4–1.7×
Backward	0.9–1.2	1.6–2.4	1.5–2.2×

Table 6: **Qwen3-8B on eight B200 GPUs.** TP-8 with sequence parallelism, micro-batch two. Memory columns use SwiGLU recomputation. The BF16 storage baseline expands K/V to 32 query heads. QRO includes packed attention state; additional packing covers working weights and non-attention factors. † exceeds 167.6 GiB. At 512K, $*$ marks full-checkpoint BF16 timing, with 95.2 GiB memory.

	Step rate (k tokens/s)			Memory/GPU (GiB)		
Context	BF16	FP4	Speedup	BF16, expanded K/V	QRO	+ packed
32,768	105	168	1.60×	36.5	28.6	23.7
131,072	53	80	1.50×	84.6	53.3	37.7
262,144	32	47	1.46×	148.9	86.1	56.3
524,288	14^{*}	26	1.79×	$277.3^{\dagger}$	151.9	93.6

Larger-model capacity. On 8xRTX PRO 6000 GPUs, QRO fits 13B at 262,144 tokens & 32B at 65,536 tokens within a 95 GiB device budget. The 32B at 131,072 tokens exceeds capacity (Table 7).

Table 7: **Larger models on eight RTX PRO 6000 GPUs.** TP-8, micro-batch one, BF16 residuals, no activation checkpointing. Totals are GiB per GPU; † exceeds the 95 GiB budget.

Model	Context	BF16	QRO
13B	131,072	61.8	42.2
13B	262,144	$99.4^{\dagger}$	60.2
32B	65,536	$107.7^{\dagger}$	82.6
32B	131,072	$155.9^{\dagger}$	$105.6^{\dagger}$

5.5 Numerical error diagnostics

On frozen Qwen3-8B tensors, 10^4 two-operand SR draws yield a relative mean-bias bound below 0.003 at 3σ, consistent with Theorem 1. The squared-error fit has slope 1.001 and $R^2 = 0.983$; propagated noise accounts for 62.8% of query-gradient squared error (Appendix C.2). Exact one- and two-key checks reproduce the error identity within 6×10^{-17}.

Frozen-row sweeps show stable forward probability-mass loss through 131,072 tokens and stable diffuse-row backward error through 65,536 tokens. Forward output error decreases with context at fixed sink mass (Appendix C.3).

6 Conclusion

CroissAttn combines complete FP4 attention backward, a conditional-mean target, and query-row-owned execution. Its error model separates propagated and fresh rounding. Training and long-context benchmarks characterize the quality and systems behavior of the complete path.

Limitations. The conditional-mean guarantee concerns represented products under Theorem 1; forward quantization and finite arithmetic add error. Quality uses 2,048-token contexts and resource-limited replication (Section 5.1). E4 measures longer-context forward and per-row backward errors on frozen synthetic tensors therefore, long-context training quality remains as future work.

GENERATIVE AI USE STATEMENT

Generative AI tools were used to assist with manuscript editing and writing, primarily in the Introduction and Conclusion, including improving clarity, organization, and phrasing. Generative AI was also used to assist with the presentation of figures and plots, including chart layout, labels, legends, and visual readability. All technical content, mathematical claims, experimental results, numerical values, and scientific conclusions are the authors' own.

ETHICS STATEMENT

This work studies numerical precision for language-model training. It uses publicly released model checkpoints (Qwen3-1.7B and Qwen3-8B), the public FineWeb-Edu corpus, and public zero-shot benchmarks, all as released and without modification. It involves no human subjects, no new data collection, and no personally identifying information beyond what those public corpora already contain. The method changes how attention is computed and stored during training; it adds no new model capability, and the trained checkpoints are evaluated only on standard benchmarks. Lower-precision training reduces the compute, memory, and energy cost of training language models, and that reduction applies to all uses of such models. We evaluate quality with validation loss and seven zero-shot tasks; we do not evaluate whether FP4 training or FP4 serving changes safety-relevant model behavior.

REPRODUCIBILITY STATEMENT

Section 4 specifies the method: the quantized forward and its straight-through target (Equations (1) and (2)), the two-operand stochastic-rounding backward with auxiliary correction (Equations (4) and (5)), the saved-factor layouts (Table 1), the query-row-owned schedule (Algorithm 1), and the covering-scale rule (Equation (8)). Appendix B.2 gives the replay contract and the fields of the logical random identifier, and Appendix B.3 gives range handling for streamed operands. Appendix A proves Theorems 1 and 2 and the supporting product identities. Section 5.1 and Appendix E give the precision recipe and rounding policy of every training arm, the continued-training schedules, batch sizes, and master-weight and optimizer-state precision, the from-scratch budgets and pairing, and the hardware for each study.

REFERENCES

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://ojs.aaai.org/index.php/AAAI/article/view/6239.

Roberto Castro, Andrei Panferov, Rush Tabesh, Oliver Sieberling, Jiale Chen, Mahdi Nikdan, Saleh Ashkboos, and Dan Alistarh. Quartet: Native fp4 training can be optimal for large language models. *Advances in Neural Information Processing Systems*, 38:43552–43572, 2026.

Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. FP4 all the way: Fully quantized training of large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=kuzye4EPLR.

Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1300. URL https://aclanthology.org/N19-1300/.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

Matteo Croci, Massimiliano Fasi, Nicholas J. Higham, Théo Mary, and Mantas Mikaitis. Stochastic Rounding: Implementation, Error Analysis and Applications. *Royal Society Open Science*, 9(3): 211631, 2022. doi: 10.1098/rsos.211631.

Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *ICLR*, 2024. URL https://arxiv.org/abs/2307.08691.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html. arXiv:2205.14135.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2208.07339.

Siyu Ding, Mingchuan Ma, Jiabo Tong, Xingrun Xing, Ziming Wang, and Guoqi Li. Full-Stack FP4: Stable LLM Pretraining with Quantized Projections, Optimizers, and Attention. arXiv:2607.04422v2, 2026. URL https://arxiv.org/abs/2607.04422.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *ICLR*, 2019. URL https://arxiv.org/abs/1711.05101.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed Precision Training. In *ICLR*, 2018. URL https://arxiv.org/abs/1710.03740.

Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. FP8 Formats for Deep Learning. arXiv:2209.05433, 2022. URL https://arxiv.org/abs/2209.05433.

NVIDIA. Pretraining Large Language Models with NVFP4. arXiv:2509.25149v2, 2025. URL https://arxiv.org/abs/2509.25149.

NVIDIA. Parallel Thread Execution ISA: Block-Scaled Matrix Instructions. CUDA documentation, 2026. URL https://docs.nvidia.com/cuda/parallel-thread-execution/.

Andrei Panferov, Erik Schultheis, Soroush Tabesh, and Dan Alistarh. Quartet II: Accurate LLM Pre-Training in NVFP4 by Improved Unbiased Gradient Estimation. arXiv:2601.22813v2, 2026. URL https://arxiv.org/abs/2601.22813.

Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1525–1534. Association for Computational Linguistics, 2016. doi: 10.18653/v1/P16-1144. URL https://aclanthology.org/P16-1144/.

Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

Qwen Team. Qwen3: Model Cards. Official model documentation, 2025a. URL https://huggingface.co/Qwen/Qwen3-1.7B.

Qwen Team. Qwen3-8B: Model Card. Official model documentation, 2025b. URL https://huggingface.co/Qwen/Qwen3-8B.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399. URL https://ojs.aaai.org/index.php/AAAI/article/view/6399.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.

Xiao Sun, Naigang Wang, Chia-Yu Chen, Jiamin Ni, Ankur Agrawal, Xiaodong Cui, Swagath Venkataramani, Kaoutar El Maghraoui, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Ultra-Low Precision 4-bit Training of Deep Neural Networks. In *NeurIPS*, 2020. URL https://papers.nips.cc/paper_files/paper/2020/hash/13b919438259814cd5be8cb45877d577-Abstract.html.

Ruizhe Wang, Yeyun Gong, Xiao Liu, Guoshuai Zhao, Ziyue Yang, Baining Guo, Zhengjun Zha, and Peng Cheng. Optimizing Large Language Model Training Using FP4 Quantization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 62937–62957, 2025. URL https://proceedings.mlr.press/v267/wang25ae.html.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *ICML*, 2023. URL https://arxiv.org/abs/2211.10438.

Ted Zadouri, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling. arXiv:2603.05451, 2026. URL https://arxiv.org/abs/2603.05451.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472/.

Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia wei, Jun Zhu, and Jianfei Chen. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=nC8XliUxeg.

Jintao Zhang, Haoxu Wang, Pengle Zhang, Xiaoming Xu, Haofeng Huang, Kai Jiang, Jianfei Chen, Jun Zhu, et al. Sageattention3: Microscaling fp4 attention for inference and an exploration of 8-bit training. *Advances in Neural Information Processing Systems*, 38:53901–53931, 2026a.

Peiyuan Zhang, Matthew Noto, Wenxuan Tan, Chengquan Jiang, Will Lin, Wei Zhou, and Hao Zhang. Attn-QAT: 4-Bit Attention With Quantization-Aware Training. arXiv:2603.00040v3, 2026b. URL https://arxiv.org/abs/2603.00040v3.

Appendix guide. Appendix A proves the product and complete-backward error identities. Appendix B specifies the forward surrogate, replay, normalization, and finite arithmetic. Appendix C studies row correction, the E1 frozen-model budget and E4 forward/backward context sweeps, grouped-query reduction, and projection smoothing. Appendix D gives training protocols, seed-level results, and downstream evaluations. Appendix E gives storage components and the remaining systems settings.

A Product and Complete-Backward Guarantees

A.1 Conditioning and scalar moments

The product identities condition on finite inputs, layouts, and represented scales, denoted $\mathcal{G}$. The random variables are the subsequent payload draws. The attention identities condition on $\mathcal{F}_0$, containing $Q_f, K_f, V_f, U_f, P, P_f, L$, masks, and incoming dO. Later product identities additionally condition on their realized operands. All moment identities in this appendix use exact arithmetic on decoded operands.

For scale $s > 0$, the E2M1 decoded levels are $s\{0, \pm 1/2, \pm 1, \pm 3/2, \pm 2, \pm 3, \pm 4, \pm 6\}$. Define $x_c = \min(6s, \max(-6s, x))$. For adjacent levels $a \le x_c \le b$, choose b with probability $(x_c - a)/(b - a)$ and a otherwise. An exactly represented value is returned deterministically.

Lemma 3 (Scalar moments)**.** *The rounded scalar satisfies*

$$\mathbb{E}\widehat{x} = x_c, \qquad \mathrm{Var}(\widehat{x}) = (x_c - a)(b - x_c), \qquad \mathbb{E}(\widehat{x} - x)^2 = (x_c - x)^2 + (x_c - a)(b - x_c). \tag{11}$$

Proof. Writing $\pi = (x_c - a)/(b - a)$ gives $(1-\pi)a + \pi b = x_c$ and $(1-\pi)(a - x_c)^2 + \pi(b - x_c)^2 = (x_c - a)(b - x_c)$. Expanding around the mean gives the last identity. □

Nearest rounding has its decoded value as its mean and zero variance. Thus the same formulas support stochastic rounding, deterministic recasts, and mixtures of the two. Covering scales give $x_c = x$ and remove the scalar clipping bias.

A.2 Product error and covariance

Let $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{k \times n}$. Write $\widehat{A} = \bar{A} + \varepsilon_A$ and $\widehat{B} = \bar{B} + \varepsilon_B$, with centered entries mutually independent conditional on $\mathcal{G}$. Set $(\mathsf{V}_A)_{i\ell} = \mathbb{E}(\varepsilon_A)_{i\ell}^2$ and $(\mathsf{V}_B)_{\ell j} = \mathbb{E}(\varepsilon_B)_{\ell j}^2$. The centered product is

$$Z = \widehat{A}\widehat{B} - \bar{A}\bar{B} = \varepsilon_A \bar{B} + \bar{A}\varepsilon_B + \varepsilon_A \varepsilon_B. \tag{12}$$

Lemma 4 (Output covariance)**.** *For output indices (i, j) and (u, v),*

$$\begin{aligned}\mathrm{Cov}(Z_{ij}, Z_{uv}) = \mathbf{1}_{i=u} \sum_{\ell} (\mathsf{V}_A)_{i\ell} \bar{B}_{\ell j} \bar{B}_{\ell v} \\ + \mathbf{1}_{j=v} \sum_{\ell} \bar{A}_{i\ell} \bar{A}_{u\ell} (\mathsf{V}_B)_{\ell j} + \mathbf{1}_{i=u}\mathbf{1}_{j=v} \sum_{\ell} (\mathsf{V}_A)_{i\ell} (\mathsf{V}_B)_{\ell j}.\end{aligned} \tag{13}$$

Proof. Expand the products in Equation (12). For the first term, $\mathbb{E}(\varepsilon_A \bar{B})_{ij} (\varepsilon_A \bar{B})_{uv} = \sum_{\ell,t} \bar{B}_{\ell j} \bar{B}_{tv} \mathbb{E}(\varepsilon_A)_{i\ell} (\varepsilon_A)_{ut}$. Only $i = u$ and $\ell = t$ survive. The second term has the corresponding condition $j = v$, $\ell = t$. The third term needs both output indices to agree and contributes the product of variances. Each mixed term contains an independent centered error appearing once, so its expectation is zero. □

Taking $i = u$, $j = v$ yields $\mathbb{E}Z^{\odot 2} = \mathsf{V}_A \bar{B}^{\odot 2} + \bar{A}^{\odot 2} \mathsf{V}_B + \mathsf{V}_A \mathsf{V}_B$. Adding the deterministic difference $\bar{A}\bar{B} - AB$ proves Equation (9). Summing its entries gives expected squared Frobenius error. For any fixed linear consumer C, with $e = \mathrm{vec}(\widehat{A}\widehat{B} - AB)$, mean b, and covariance Σ,

$$\mathbb{E}\|Ce\|_2^2 = \|Cb\|_2^2 + \mathrm{tr}(C\Sigma C^\top). \tag{14}$$

This identity follows by writing $e = b + (e - b)$ and expanding the squared norm.

Dependence and random scales. A shared block scale is fixed in the conditional calculation. If the scales are random, total covariance gives $\mathrm{Cov}(e) = \mathbb{E}\,\mathrm{Cov}(e \mid \mathcal{G}) + \mathrm{Cov}(\mathbb{E}[e \mid \mathcal{G}])$. Correlated operand streams change the product mean by $\Gamma_{ij} = \sum_\ell \mathrm{Cov}(\widehat{A}_{i\ell}, \widehat{B}_{\ell j} \mid \mathcal{G})$. These expressions specify the covariance to use when rounding streams are shared. Within-operand independence is needed for Equation (13); the product mean needs only unbiased scalar means and independence between the multiplied operands.

A.3 The complete conditional mean

Two-operand SR gives

$$\mathbb{E}[\mathcal{M}(A, B) \mid A, B, \mathcal{G}] = AB. \tag{15}$$

For dV, apply this identity to $U_f^\top$ and $L^{-1}dO$. For dP, it gives $\mathbb{E}[\widehat{dP} \mid \mathcal{F}_0] = dOV_f^\top = dP_\star$. The auxiliary correction $D_\star$ is fixed under $\mathcal{F}_0$, hence

$$\mathbb{E}[\widehat{dS} \mid \mathcal{F}_0] = P \odot (dP_\star - D_\star)/\sqrt{d} = dS_\star. \tag{16}$$

At the query product, fresh rounding has the local mean $\mathbb{E}[\widehat{dQ} \mid \mathcal{F}_0, \widehat{dS}] = \widehat{dS}K_f$. Taking expectation gives $dS_\star K_f$. Transposing the score gradient gives the key-gradient proof. This establishes Theorem 1, including stochastic recasts of every saved factor.

Sampled centering uses $\widehat{dS} = J_P(\widehat{dP})/\sqrt{d}$. Linearity of J_P gives the same conditional mean. For each row, $\sum_j p_j(\widehat{u}_j - \sum_\ell p_\ell \widehat{u}_\ell) = 0$, so this rule also has zero row sum in every draw. The implemented auxiliary correction has the same zero row sum in expectation.

A.4 The full error decomposition

Put $S_\star = dS_\star$, $\widehat{S} = \widehat{dS}$, and $\Xi = \widehat{dP} - dP_\star$. Under Theorem 1, $\mathbb{E}[\Xi \mid \mathcal{F}_0] = 0$ and $\widehat{S} = S_\star + \mathcal{A}(\Xi)$, where $\mathcal{A}(\Xi) = P \odot \Xi/\sqrt{d}$ for auxiliary correction and $J_P(\Xi)/\sqrt{d}$ for sampled centering. Define fixed maps

$$T_Q \operatorname{vec}\Xi = \operatorname{vec}(\mathcal{A}(\Xi)K_f), \qquad T_K \operatorname{vec}\Xi = \operatorname{vec}(\mathcal{A}(\Xi)^\top Q_f). \tag{17}$$

Let $\Sigma_P = \mathrm{Cov}(\operatorname{vec}\Xi \mid \mathcal{F}_0)$. With $r(A, B) = \mathbb{E}[\|\mathcal{M}(A, B) - AB\|_F^2 \mid \mathcal{F}_0, A, B]$, the complete sum is

$$\begin{aligned}
&\mathbb{E}\Big[\|\widehat{dV} - dV_\star\|_F^2 + \|\widehat{dQ} - dQ_\star\|_F^2 + \|\widehat{dK} - dK_\star\|_F^2 \mid \mathcal{F}_0\Big] \\
&\quad = r(U_f^\top, L^{-1}dO) + \mathrm{tr}\big[(T_Q^\top T_Q + T_K^\top T_K)\Sigma_P\big] + \mathbb{E}\Big[r(\widehat{S}, K_f) + r(\widehat{S}^\top, Q_f) \mid \mathcal{F}_0\Big].
\end{aligned} \tag{18}$$

To prove this, set $\eta_Q = \widehat{dQ} - \widehat{S}K_f$. Then $\widehat{dQ} - dQ_\star = \mathcal{A}(\Xi)K_f + \eta_Q$ and $\mathbb{E}[\eta_Q \mid \mathcal{F}_0, \widehat{S}] = 0$. The conditional cross term vanishes. Equation (14) evaluates the propagated term. Repeat for dK and add the single-product dV risk.

The final risks are evaluated on each sampled score gradient before averaging. This retains changes in the selected scales and quantization neighbors. Independence between the final query and key products is unnecessary for the sum of their separate squared errors. A metric mixing the two gradients uses their joint covariance as well.

A.5 Deterministic saved-factor recasts

For gradient-only SR, denote the decoded saved factors used by backward as $U_f + \Delta_U$, $V_f + \Delta_V$, $K_f + \Delta_K$, and $Q_f + \Delta_Q$. These deterministic differences are fixed under $\mathcal{F}_0$. Put $B_X = \mathbb{E}[\widehat{dX} \mid \mathcal{F}_0] - dX_\star$. With exact auxiliary correction,

$$\begin{aligned}
&B_V = \Delta_U^\top L^{-1}dO, && B_P = dO\Delta_V^\top, && B_S = P \odot B_P/\sqrt{d}, \\
&B_Q = B_S K_f + (dS_\star + B_S)\Delta_K, && B_K = B_S^\top Q_f + (dS_\star + B_S)^\top \Delta_Q.
\end{aligned} \tag{19}$$

For sampled centering, replace B_S with $J_P(B_P)/\sqrt{d}$. These identities follow by expanding, for example, $\mathbb{E}\widehat{dQ} = (dS_\star + B_S)(K_f + \Delta_K)$, and subtracting the target.

The squared-error formula is centered at these shifted means. Let $\bar{K} = K_f + \Delta_K$, $\Xi_0 = \widehat{dP} - \mathbb{E}\widehat{dP}$, and $\widetilde{r}_Q(S) = \mathbb{E}[\|\mathcal{M}_{\mathrm{grad}}(S;\bar{K}) - S\bar{K}\|_F^2 \mid \mathcal{F}_0, S]$, where $\mathcal{M}_{\mathrm{grad}}$ rounds only S and holds the decoded recast $\bar{K}$ fixed. Then

$$\mathbb{E}\|\widehat{dQ} - dQ_\star\|_F^2 = \|B_Q\|_F^2 + \mathbb{E}\|\mathcal{A}(\Xi_0)\bar{K}\|_F^2 + \mathbb{E}\widetilde{r}_Q(\widehat{S}). \tag{20}$$

The deterministic bias, centered propagated error, and fresh conditionally centered error have zero expected cross terms. The key identity follows by transposition. Thus nearest recasts are handled by a bias term and by propagating noise through their actual decoded means.

B Forward Surrogate and Execution Details

B.1 Where the straight-through derivative is placed

Fix a query row and write $m = \max_j s_j$, $U_j = e^{s_j - m}$, $\ell = \sum_j U_j$, $p = U/\ell$, and $p_f = U_f/\ell$. The forward output is $o = V_f^\top p_f$. For incoming gradient h, put $u = V_f h$, $D_\star = p^\top u$, $D_O = p_f^\top u$, and $r = D_O - D_\star$. Quantizer scales and payload draws are fixed during differentiation.

Normalized-probability surrogate. For $p_{\mathrm{STE}} = p + \mathrm{sg}(p_f - p)$, the differential is $\delta p_{\mathrm{STE}} = J_p \delta s$. Thus

$$g_{\mathrm{norm}} = J_p u, \qquad J_p = \mathrm{diag}(p) - pp^\top, \qquad \mathbf{1}^\top g_{\mathrm{norm}} = 0. \tag{21}$$

Its value-factor gradient is $p_f h^\top$. This gives Equation (2).

Identity STE on the numerator. Applying the identity derivative directly to $\mathcal{Q}(U)$ with m fixed gives $\delta U_j = U_j \delta s_j$ and $\delta \ell = \sum_j U_j \delta s_j$. Differentiating $h^\top V_f^\top U_f/\ell$ yields

$$g_{\mathrm{fixed}} = p \odot (u - D_O \mathbf{1}) = g_{\mathrm{norm}} - rp, \qquad \mathbf{1}^\top g_{\mathrm{fixed}} = -r. \tag{22}$$

For a differentiated maximum with derivative a satisfying $\mathbf{1}^\top a = 1$, $\delta m = a^\top \delta s$ gives

$$g_{\max} = g_{\mathrm{fixed}} + ra = g_{\mathrm{norm}} + r(a - p), \qquad \mathbf{1}^\top g_{\max} = 0. \tag{23}$$

A unique maximizing key uses $a = e_k$; a tie uses the selected subgradient supported on the maxima. These formulas specify three derivatives for the same forward values. CroissAttn uses Equation (21).

Auxiliary correction error. The normalized target uses $D_\star = \langle h, V_f^\top p \rangle$. Suppose $\mathbb{E}\widehat{u} = u + b_u$ and the available correction is $\widetilde{D} = D_\star + r_D$. The score-gradient bias is

$$\mathbb{E}\Big[\mathrm{diag}(p)(\widehat{u} - \widetilde{D}\mathbf{1})/\sqrt{d}\Big] - J_p u/\sqrt{d} = \big(\mathrm{diag}(p) b_u - r_D p\big)/\sqrt{d}. \tag{24}$$

This follows by linearity. If $\widetilde{o}' = o' + e_o$, then $r_D = h^\top e_o$ and the correction-only bias norm is bounded by $\|h\|_2 \|e_o\|_2 \|p\|_2/\sqrt{d}$. Using the actual forward output gives $r_D = D_O - D_\star$ and the fixed-maximum target in Equation (22). FlashAttention uses $D_i = \langle dO_i, O_i \rangle$ for its forward output $O = PV$ (Dao et al., 2022, Supplement, Algorithm 4, line 19). Attn-QAT introduces the auxiliary $O' = PV_f$ for quantized attention (Zhang et al., 2026b, Section 2.3, Equation 9, Algorithms 2–3); Equation (4) uses that construction for our normalized-probability target.

B.2 Normalization and replay

The identity $P_f^\top dO = U_f^\top (L^{-1} dO)$ puts normalization on the incoming gradient before its payload draw. Conditional on $\mathcal{F}_0$, covering SR gives $\mathbb{E}\,\mathrm{SR}(L^{-1}dO) = L^{-1}dO$. An independent SR recast of $U_f^\top$ then gives the value-gradient mean. A nearest numerator recast contributes exactly $\Delta_U^\top L^{-1} dO$.

Replay contract. A deterministic sequence of machine operations reproduces its bits when its input bits, instruction choices, masks, reduction order, rounding modes, scales, and stochastic counters agree. The proof is induction over the operation sequence. Score replay uses this contract in the original contraction layout. Saved factors consumed along a new contraction axis use the recasts described in Section 4.2.

A logical random identifier includes the training step, micro-batch, layer, product site, operand, batch item, head, and tensor coordinates. Replaying a stochastic operation reuses its original identifier; a fresh recast uses a new product-site identifier. Reusing saved payloads requires no new draw. Transposed views map back to the chosen logical coordinates. Scale-block membership is part of the quantization call. This description separates repeated evaluation of one draw from a new stochastic representation.

Probability replay perturbations. For two probability rows p, q over the same keys,

$$\|J_p - J_q\|_2 \le 3\|p - q\|_2. \tag{25}$$

Indeed, with $\delta = p - q$, $J_p - J_q = \operatorname{diag}(\delta) - p\delta^\top - \delta q^\top$. Each term has norm at most $\|\delta\|_2$ because probability vectors have Euclidean norm at most one. For fixed u, the corresponding score-gradient difference is at most $3\|p - q\|_2\|u\|_2/\sqrt{d}$.

B.3 Covering streamed operands

For a finite tile X, decoded values have the form $g_X b_B c$, with $|c| \le 6$ and positive E4M3 scale $b_B \le 448$. Choose a finite representable FP32 multiplier satisfying Equation (8). Round the required block scale upward to a supported positive E4M3 value. This gives $\max_{x\in B}|x| \le 6 g_X b_B$ and $b_B \le 448$. An all-zero tile can use a safe positive scale and zero payloads.

The implementation check uses the decoded scales actually supplied to the instruction. A nonzero block must retain a positive scale, including at the smallest supported magnitude. A block requiring $b_B > 448$ triggers a larger multiplier and new scale selection before rounding. Nonfinite inputs or values beyond the supported covering range require an explicit error or a specified fallback. Fixed and delayed multipliers use the same current-tile covering check.

If a contraction is split into tiles t, accumulate

$$C = \sum_t (g_{A,t} g_{B,t})\,\mathrm{MMA}(b_{A,t}c_{A,t}, b_{B,t}c_{B,t}). \tag{26}$$

Each local multiplier is applied to its own contribution. The scale selection for $\widehat{dS}$ uses its realized values after probability-gradient rounding and row correction. This preserves the conditioning used in the final-product risks of Theorem 2.

B.4 Forward-target and finite-arithmetic error

Let $g_\star$ be the target in Equation (2), g_{ref} a fixed BF16 reference gradient, $\widehat{g}$ the represented backward, and $\widetilde{g}$ its implemented result. Then

$$\widetilde{g} - g_{\mathrm{ref}} = \underbrace{g_\star - g_{\mathrm{ref}}}_{b_{\mathrm{fwd}}} + \underbrace{\mathbb{E}\widehat{g} - g_\star}_{b_{\mathrm{bwd}}} + \underbrace{\widehat{g} - \mathbb{E}\widehat{g}}_{\xi} + \underbrace{\widetilde{g} - \widehat{g}}_{e_{\mathrm{exec}}}. \tag{27}$$

Theorem 1 removes b_{bwd} under its assumptions. The forward-target term includes the selected surrogate and represented factors. For example, fixing Q_f, K_f, V_f, dO and changing only the output coefficients from P to P_f gives $(P_f - P)^\top dO$ in the value gradient.

For reference factors Q_0, K_0, V_0, write $P_0 = \mathrm{softmax}(Q_0K_0^\top/\sqrt{d})$ and $\Delta_V = V_f - V_0$, with analogous query/key differences. Exact subtraction gives

$$\begin{aligned} dS_\star - dS_0 &= \big[(J_P - J_{P_0})(dOV_0^\top) + J_P(dO\Delta_V^\top)\big]/\sqrt{d}, \\ dQ_\star - dQ_0 &= (dS_\star - dS_0)K_0 + dS_\star\Delta_K, \\ dK_\star - dK_0 &= (dS_\star - dS_0)^\top Q_0 + dS_\star^\top\Delta_Q. \end{aligned} \tag{28}$$

These local identities hold at the fixed incoming dO.

For a fixed linear consumer C, if $\mathbb{E}\|Ce_{\mathrm{exec}}\|_2^2 \le \epsilon_{\mathrm{exec}}^2$, the reverse triangle inequality in the space of square-integrable random vectors gives

$$\left|\sqrt{\mathbb{E}\|C(\widetilde{g} - g_\star)\|_2^2} - \sqrt{\mathbb{E}\|C(\widehat{g} - g_\star)\|_2^2}\right| \le \epsilon_{\mathrm{exec}}. \tag{29}$$

If the represented backward is unbiased, Jensen's inequality also gives $\|\mathbb{E}C(\widetilde{g} - g_\star)\|_2 \le \epsilon_{\mathrm{exec}}$.

C Numerical Diagnostics and Supporting Analysis

C.1 How does row correction change propagated noise?

The training and systems experiments use auxiliary correction. This subsection compares its error map with sampled centering on fixed tensors. For a row with probability vector p, centered probability-gradient noise ξ, covariance Σ, and fixed downstream map C, the two propagated risks are

$$R_{\text{aux}} = \frac{1}{d}\operatorname{tr}\big(C\operatorname{diag}(p)\Sigma\operatorname{diag}(p)C^\top\big), \qquad R_{\text{sampled}} = \frac{1}{d}\operatorname{tr}(CJ_p\Sigma J_p C^\top). \tag{30}$$

Their score-gradient errors differ by $p(p^\top\xi)/\sqrt{d}$. Sampled centering removes the component that is common to all keys because $J_p\mathbf{1} = 0$. In a one-key row, $J_p = 0$, so its propagated error is zero in every draw.

Incoming-gradient covariance. Hold saved values exact and use the same rounded incoming gradient $\widehat{g} = g + \varepsilon$ across keys. Let $\bar{k} = \sum_j p_j k_j$, $\bar{v} = \sum_j p_j v_j$, and $M_{kv} = \sum_j p_j k_j v_j^\top$. The two query-gradient perturbations are

$$\delta q_{\text{aux}} = M_{kv}\varepsilon/\sqrt{d}, \qquad \delta q_{\text{sampled}} = (M_{kv} - \bar{k}\bar{v}^\top)\varepsilon/\sqrt{d}. \tag{31}$$

Substitution of $\xi_j = v_j^\top\varepsilon$ into Equation (30) proves this expression. The second map is the attention-weighted key/value covariance. Stochastic saved-value recasts contribute additional terms through Equation (13).

Exact examples. Table 8 enumerates E2M1 rounding in one- and two-key examples, including saved-factor recasts and sample-adaptive final-product scales. The direct totals and the propagated-plus-fresh sums agree within 6×10^{-17}. Sampled centering gives zero error in the one-key case and reduces the two-key total from 0.21037027 to 0.01670725.

Table 8: **Exact causal checks.** Query-plus-key squared error for one query, head dimension one. Each total sums propagated and fresh-product error.

Keys	Correction	Propagated	Fresh	Total
1	Auxiliary	0.03356934	0.00253042	0.03609975
1	Sampled	0	0	0
2	Auxiliary	0.19592679	0.01444348	0.21037027
2	Sampled	0.01013594	0.00657131	0.01670725

Attention concentration. A second diagnostic uses causal Gaussian tensors at $T = 256$, $d = 64$, and 400 incoming-gradient SR draws with fixed saved factors. Table 9 groups rows by their location and reports the effective number of attended keys, $n_{\text{eff}} = 1/\sum_j p_j^2$. The auxiliary-to-sampled variance ratio decreases from 1.98 to 1.14 across the multi-key groups. This behavior describes the shared-noise component removed by sampled centering as attention becomes more diffuse.

Table 9: **Row correction versus attention concentration.** Per-row query-gradient variance from incoming-gradient rounding. Row indices are zero-based.

Query rows	n_{eff}	Auxiliary	Sampled	Variance ratio
0	1.00	0.946	0.000	∞
1–3	2.23	0.495	0.250	1.98
4–15	6.37	0.212	0.143	1.48
16–63	18.3	0.089	0.070	1.26
64–255	63.7	0.025	0.022	1.14

The consumer and covariance determine the ordering in Equation (30). For example, take $d = 1$, $p = (1/2, 1/2)^\top$, $K_f = (0, 1)^\top$, and $\xi = (\varepsilon, 0)^\top$ with unit-variance centered ε. Auxiliary correction gives $K_f^\top \operatorname{diag}(p)\xi = 0$; sampled centering gives $K_f^\top J_p\xi = -\varepsilon/4$, with variance $1/16$. This example characterizes the dependence on the downstream map.

C.2 Does the propagated-error model match frozen tensors?

The E1 study freezes Qwen3-8B tensors after 200M continued-training tokens. It uses 16 sequences of length 2,048, all heads, and layers $\{4, 12, 20, 28, 36\}$ with *one-based layer indexing*. Forward factors, probabilities, and incoming loss gradients are fixed. E1 uses exact auxiliary correction and two-operand SR for gradient operands and saved-factor recasts, with one seed generating 10^4 backward draws. Represented products are evaluated in FP32, with FP64 products of the same operands as execution references.

Table 10 consolidates the pooled summary. The error fit uses one point for each layer, head, and returned gradient, giving $5 \times 32 \times 3 = 480$ points. The slope of 1.001 and $R^2 = 0.983$ describe agreement between sampled and predicted squared error. The pooled query-gradient propagated share is 0.628. The relative mean-bias bound is below 0.003 at 3σ over 10^4 draws, supporting the conditional-mean prediction. Conditional fresh-product risks are evaluated on their sampled operands before averaging.

Table 10: **Pooled frozen-tensor error summary.** Relative errors are averaged over heads and five layers; gradient norms use the specified STE target. GQA identifies the shared-head sum. The propagated share is a fraction of squared error. E1 uses two-operand SR; the bias bound uses 10^4 draws.

Quantity	Value
dP relative error	0.170
dS relative error	0.190
dQ relative error	0.250
dK relative error, GQA sum	0.160
dV relative error, GQA sum	0.110
Propagated share of dQ error	0.628
Sampled/predicted squared-error slope	1.001
R^2 over layer/head/gradient points	0.983
Relative mean-bias bound (3σ, 10^4 draws)	< 0.003
FP32 execution discrepancy versus FP64	2.3×10^{-6}

A synthetic study uses causal attention with $T = 256$, $d = 64$, outlier-heavy channels, three heads, five attention temperatures, and 500 draws per configuration. Across the 15 configurations, sampled/predicted squared-error ratios range from 0.980 to 1.010. The propagated share of query-gradient error ranges from 0.30 to 0.63. These values describe the synthetic temperature sweep.

C.3 E4: Does the error budget depend on context length?

E4 evaluates frozen synthetic rows on CPU, without training or GPU execution. The head dimension is 128, with three independent query rows per context and one fixed attention temperature. Keys and values are Gaussian. The *no-sink* regime uses a plain softmax row; the *fixed-logit sink* adds $+6$ to the first key's logit. A separate *fixed-mass sink* assigns the first key 40% or 70% of the normalized probability mass. Forward measurements reach 131,072 keys; backward measurements reach 65,536.

Forward representation and metrics. The forward rounds the unnormalized numerator $U_j = \exp(s_j - \max_k s_k)$ to nearest, giving $U_f = \mathcal{Q}(U)$, and applies the FP32 row sum $\ell = \sum_j U_j$ afterwards. Thus $p = U/\ell$ and $p_f = U_f/\ell$. The input to the FP4 quantizer has maximum one at every context length. Growth of ℓ does not shrink that input; normalization acts after payload rounding.

Let $\mathcal{Z} = \{j : (U_f)_j = 0,\ p_j > 0\}$. Lost probability mass is $\sum_{j\in\mathcal{Z}} p_j$, and the largest lost probability is $\max_{j\in\mathcal{Z}} p_j$. For one output row, the probability-only error compares $V_f^\top p_f$ with $V_f^\top p$, using the same represented values. The total error compares with fully unquantized attention and includes the query, key, and value casts. Both output errors are relative Euclidean errors against their respective references. Forward rounding is deterministic, so Table 11 uses no stochastic-rounding draws.

Forward error across contexts. In no-sink rows, lost mass stays at 1.7–2.0% across a 256-fold context span, and probability-only output error stays at 0.087–0.098. The largest probability rounded

Table 11: **E4a: Forward probability-rounding cost.** (a) No-sink and fixed-+6-logit rows. Zeroed entries are percentages; lost mass is normalized probability mass. Output errors are relative norms. (b) Fixed-mass controls, with $n_{\text{eff}} = 1/\sum_j p_j^2$. The no-sink endpoint from (a) is repeated for comparison.

(a) No sink and fixed-logit sink

Regime	T	Zeroed entries	Lost mass	Largest p lost	O error P only	O error total
No sink	512	9.6%	1.7e−2	7.9e−4	8.8e−2	1.6e−1
No sink	2,048	12.1%	1.9e−2	3.4e−4	8.7e−2	1.7e−1
No sink	8,192	11.3%	1.9e−2	1.2e−4	9.8e−2	2.0e−1
No sink	32,768	10.2%	1.8e−2	5.0e−5	9.0e−2	1.9e−1
No sink	131,072	11.7%	2.0e−2	1.9e−5	9.7e−2	2.1e−1
Sink	512	11.9%	3.0e−2	4.1e−3	4.1e−2	1.9e−1
Sink	2,048	12.8%	1.9e−2	1.4e−3	1.8e−2	1.4e−1
Sink	8,192	11.5%	2.1e−2	5.6e−4	8.0e−2	1.6e−1
Sink	32,768	10.2%	1.8e−2	9.9e−5	5.7e−2	1.8e−1
Sink	131,072	11.7%	2.0e−2	2.4e−5	8.8e−2	2.0e−1

(b) Fixed-mass forward controls

Regime	T	n_{eff}	Lost mass	O error, P only
40% sink	512	6.2	2.6e−2	1.8e−2
40% sink	8,192	6.2	1.3e−2	3.2e−3
40% sink	131,072	6.2	**1.2e−2**	**6.6e−4**
70% sink	131,072	2.0	5.9e−3	2.0e−4
No sink	131,072	46,284	2.0e−2	9.7e−2

to zero falls from 7.9×10^{-4} to 1.9×10^{-5}, about 42-fold. Fixed-logit sink rows lose 3.0% mass at 512 keys and 1.8–2.1% at the larger contexts. These measurements show stable discarded mass under numerator quantization before normalization.

The fixed-mass controls separate context growth from sink dilution. At 40% sink mass, n_{eff} remains 6.2 at the displayed precision, and probability-only output error falls from 0.018 to 0.0032 and 0.00066, a 27.3-fold endpoint reduction. Lost mass falls from 2.6% to 1.2%. At 131,072 keys, the 70% sink has $n_{\text{eff}} = 2.0$, lost mass 0.59%, and output error 0.0002. The output error improves while attention remains concentrated. Its size depends on the value-weighted difference $V_f^\top (p_f - p)$, so discarded probability mass alone does not determine it.

Backward protocol and vector-bias statistic. Nearest-rounded forward factors are held fixed and reused at their exact decoded values. Gradient-operand SR rounds gradient operands, including dO, while saved factors are reused at their exact decoded values without recasting. For each of the three query rows, E4 uses $N = 120$ draws. Backward errors are relative to the saved-forward straight-through target in Equation (2). The dP, dS, and dQ errors and propagated shares in Table 12 use exact auxiliary correction. The auxiliary-to-sampled column compares total dQ error variance under the two correction rules.

For the 128-dimensional error $e^{(r)} = \widehat{dQ}^{(r)} - dQ_\star$, define

$$\bar{e} = \frac{1}{N}\sum_{r=1}^{N} e^{(r)}, \qquad \sigma_{\text{vec}} = \sqrt{\frac{\mathbb{E}\|e^{(1)}\|_2^2}{N}}, \qquad \frac{\text{bias}}{\sigma_{\text{vec}}} = \frac{\|\bar{e}\|_2}{\sigma_{\text{vec}}}. \tag{32}$$

Under zero mean and independent draws, $\mathbb{E}\|\bar{e}\|_2^2 = \sigma_{\text{vec}}^2$. The denominator is the root-mean-square norm of the whole-vector sample mean under that hypothesis. The statistic has root-mean-square value one; normalizing by a common per-coordinate standard error would instead give $\sqrt{128}$. The observed 0.84–1.50 range is consistent with no detectable bias at this draw budget.

No systematic growth in diffuse-row backward error. Across the 128-fold span in T, no-sink relative errors remain at 0.135–0.144 for dP, 0.132–0.150 for dS, and 0.186–0.218 for dQ;

Table 12: **E4b: Backward error versus context length.** Three independent query rows per context, 120 SR draws per row, $d = 128$. Relative errors and propagated shares use exact auxiliary correction. The variance ratio gives the frozen-tensor prediction for sampled centering. The last column uses the full-vector normalization in Equation (32).

			Relative error			Propagated	Aux./sampled	Bias/
Regime	T	n_{eff}	dP	dS	dQ	share	variance	σ_{vec}
No sink	512	197	0.144	0.150	0.215	56.2%	1.02	1.21
No sink	2,048	880	0.139	0.132	0.186	51.0%	1.01	1.16
No sink	8,192	2,912	0.135	0.136	0.206	51.8%	1.02	1.33
No sink	32,768	13,059	0.141	0.140	0.193	52.7%	1.00	1.20
No sink	65,536	25,206	0.136	0.136	0.218	51.6%	1.01	1.32
Sink	512	14.5	0.144	0.776	**0.827**	94.8%	**2.75**	0.84
Sink	2,048	52.2	0.139	0.273	0.283	89.8%	1.44	0.97
Sink	8,192	1,360	0.135	0.173	0.218	70.8%	1.17	1.50
Sink	32,768	7,566	0.141	0.176	0.214	64.1%	1.01	1.13
Sink	65,536	19,656	0.136	0.146	0.217	56.1%	1.01	1.26

propagated shares stay at 51.0–56.2%. Figure 4 shows the nearly flat query-gradient error. At $T = 2{,}048$, E4 gives $(0.139, 0.132, 0.186)$ and 51.0% propagated share. E1 gives $(0.170, 0.190, 0.250)$ and 62.8% on frozen Qwen3-8B tensors (Table 10), with two-operand SR and saved-factor recasts.

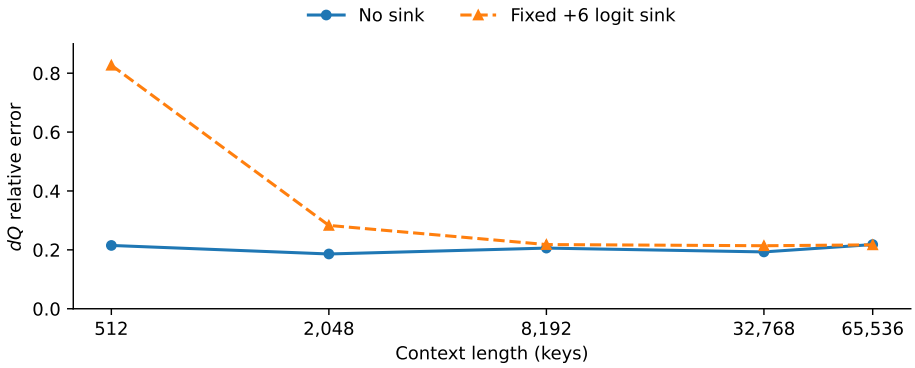


Figure 4: **E4b: Query-gradient error across contexts.** Exact auxiliary correction, gradient-operand SR, and exact saved factors. Points are the Table 12 summaries. The no-sink series stays nearly flat. The fixed-+6-logit sink becomes diluted as keys are added; fixed-mass forward controls are in Table 11(b).

Concentration and key geometry. At $T = 512$, the fixed-logit sink has $n_{\text{eff}} = 14.5$ and dQ relative error 0.827, about four times the no-sink value. As the context grows, n_{eff} rises to 19,656 and error falls to 0.217. Within this sweep, the auxiliary-to-sampled variance ratio falls from 2.75 to 1.44, 1.17, and 1.01 as attention spreads. The separate $d = 64$ Gaussian sweep in Table 9 gives a smaller ratio, 1.98, at $n_{\text{eff}} = 2.23$. Equation (30) includes the downstream key map $C = K_f^\top$ and the upstream covariance Σ alongside p. Concentration, covariance, and key geometry jointly determine the correction risks.

Prediction for sampled centering. The model predicts negligible variance reduction for diffuse rows here and up to 2.75-fold reduction for the sink-dominated rows, with the size set by their key geometry. All training and systems results use auxiliary correction; no trained sampled-centering arm tests this prediction. Propagated error accounts for 51.0–94.8% of dQ squared error here and 62.8% in E1. The 2.75 ratio already compares total variances, corresponding to $1 - 1/2.75 \approx 63.6\%$ lower

total dQ variance for those rows. A head-level benefit depends on their contribution to the head's aggregate error.

Scope. The backward sweep covers dP, dS, and dQ per row through 65,536 keys. It does not measure dK or dV, which aggregate over queries, or backward error under fixed-mass sinks. The forward tests reach 131,072 keys. Both use synthetic Gaussian factors, hand-placed sinks, one head dimension, and one temperature. Captured Qwen3-8B rows across contexts and long-context training quality are the remaining model-level tests.

C.4 How do shared-head gradients aggregate?

For query-head targets g_h sharing one key/value head, the aggregate is $g = \sum_{h=1}^{G} g_h$. If each estimate is conditionally unbiased, linearity gives $\mathbb{E}\sum_h \widehat{g}_h = \sum_h g_h$. Head independence is unnecessary for this mean identity.

For the relative-error calculation, suppose $\|g_h\|_2 = a > 0$, $\mathbb{E}\|\varepsilon_h\|_2^2 = v$, and, for $G \geq 2$, define average pairwise target cosine $\bar{c}$ and normalized error cross moment $\bar{\rho}$ by

$$\bar{c} = \frac{\sum_{h\neq k}\langle g_h, g_k\rangle}{G(G-1)a^2}, \qquad \bar{\rho} = \frac{\sum_{h\neq k}\mathbb{E}\langle \varepsilon_h, \varepsilon_k\rangle}{G(G-1)v}. \tag{33}$$

Expanding numerator and denominator of the aggregate relative RMS error gives

$$\frac{\sqrt{\mathbb{E}\|\sum_h \varepsilon_h\|_2^2}}{\|\sum_h g_h\|_2} = \frac{\sqrt{v}}{a}\sqrt{\frac{1+(G-1)\bar{\rho}}{1+(G-1)\bar{c}}}. \tag{34}$$

This expression applies when the aggregate target norm is nonzero. At $G = 1$, the relative error is $\sqrt{v}/a$. Uncorrelated head errors have $\bar{\rho} = 0$, giving the alignment factor $[1 + (G-1)\bar{c}]^{-1/2}$. For unequal head norms or variances, the unsimplified sums define the relative error directly.

The synthetic E2 check uses $T = 256$, $d = 64$, NVFP4 emulation, and 150 draws. Each head has independently drawn Q and dO, giving approximately unaligned target gradients. The group-summed relative errors remain close to 0.21 across group sizes (Table 13), consistent with the unaligned case of Equation (34).

Table 13: **Synthetic shared-head check.** Relative error of group-summed dK with shared K, V.

Rounding stream assignment	$G = 1$	$G = 4$	$G = 8$
Independent per head	0.215	0.216	0.211
One stream shared by the group	0.213	0.215	0.211

C.5 Projection smoothing and optimizer metrics

For a positive diagonal matrix D, set $\bar{X} = XD^{-1}$ and $\bar{W} = DW$. Then

$$XW = \bar{X}\bar{W}, \qquad dX = (G\bar{W}^\top)D^{-1}, \qquad dW = D(\bar{X}^\top G). \tag{35}$$

The forward identity follows from $D^{-1}D = I$. For the input derivative, use $\delta\bar{X} = \delta X D^{-1}$ in the Frobenius inner product with $G\bar{W}^\top$. For the weight derivative, use $\delta\bar{W} = D\delta W$. These yield the two restoration factors in Equation (35). Their error is obtained by applying Equation (14) to the corresponding linear restoration map.

A preceding affine channel scale $X = Z\,\mathrm{diag}(\gamma)$ can absorb D^{-1} by replacing γ with $D^{-1}\gamma$, provided every linear consumer's weight is replaced with DW_j. Each branch output is then unchanged. Training in the new parameter coordinates also transforms the gradients supplied to the optimizer.

The smoothing diagnostic in Table 14 reports product-error ratios for outlier-heavy activations. At $\alpha = 0.3$, the forward ratio is 0.572, input-gradient ratio is 1.019, and weight-gradient ratio is 0.968. The training recipe uses this exponent to balance forward and backward errors.

Table 14: **Smoothing-strength diagnostic.** Product-error ratios under the declared forward and backward rounding rules.

α	Forward	dX	dW
0.2	0.681	1.004	0.957
0.3	0.572	1.019	0.968
0.4	0.512	1.057	0.948
0.5	0.503	1.122	0.924

A fixed Adam denominator. For fixed previous first moment, denominator $d_{ij} = \sqrt{v_{ij}} + \epsilon > 0$, learning rate η, and bias-correction factor c_1, replacing a gradient G by $\widehat{G}$ changes the update by $-\eta c_1(1-\beta_1)(\widehat{G}_{ij} - G_{ij})/d_{ij}$ (Loshchilov & Hutter, 2019). Its expected squared norm is

$$\eta^2 c_1^2 (1-\beta_1)^2 \sum_{ij} \frac{\mathbb{E}(\widehat{G}_{ij} - G_{ij})^2}{d_{ij}^2}. \tag{36}$$

A common decoupled weight-decay term cancels. This is the fixed-consumer metric associated with that denominator; updating the denominator introduces the optimizer's nonlinear dependence on the new gradient.

D Training Protocols and Downstream Evaluation

D.1 Precision recipes and run organization

The experiments separate attention-only, projection-only, and complete FP4 recipes. Projection-only runs retain BF16 attention. Complete runs quantize both projection and attention products; the vocabulary readout, softmax, reductions, accumulation, master parameters, and optimizer arithmetic remain in higher precision. The smoothed complete recipe uses $\alpha = 0.3$. All quality experiments use a 2,048-token training context.

All forward FP4 operands, including Q_f, K_f, V_f and the probability numerator U_f, use nearest rounding. All FP4 training backward products use two-operand SR. This includes the 500M/1.7B from-scratch runs, the three-seed 181M study, the 511M-token attention-only pair, and the smoothed and unsmoothed continued-training arms. E1 also uses two-operand SR. E4 uses gradient-operand SR with exact saved factors to isolate the frozen-row error budget. The training and systems kernels use the auxiliary correction of Equation (4). The numerical comparisons in Appendices C.1 and C.3 evaluate both correction rules on fixed tensors. No training arm uses sampled centering.

The Qwen3-1.7B continued-training study uses FineWeb-Edu, micro-batch two, FP32 master weights, and cosine learning-rate decay from 2×10^{-5}. The seven-arm comparison in Table 2 completes 762 steps, approximately 200M tokens. Its BF16 reference is 2.4278. The BF16 and smoothed complete-FP4 continuation each reach 2,288 steps at the 600M-token horizon.

The Qwen3-8B continued-training study uses 762 steps at 262,144 tokens per step, micro-batch one, activation checkpointing, BF16 master weights and gradients, BF16 Adam first- and second-moment storage, and peak learning rate 2×10^{-5} on one RTX PRO 6000 Blackwell GPU. Both arms use the same optimizer configuration. The FP4 arm uses native FP4 instructions for projection and attention products. The training-curve endpoint is 2.1278 for BF16 and 2.1839 for FP4, a gap of 0.056 nats. The main-text curve in Figure 3 shows the recorded validation points.

Replication. GPU resource constraints limited the 181M/100M-token study to three paired training seeds and the remaining training experiments to one seed per arm. The stochastic-rounding draws and independent query rows in the frozen-tensor diagnostics are numerical samples, separate from training-seed replication.

From-scratch budgets. The 500M and 1.7B models train with native FP4 projection and attention products on B200. Each comparison pairs initialization and data order and uses one seed per arm. Table 3 gives the complete-FP4 validation gaps at 1B and 10B tokens. At 10B tokens, the smoothed

projection-only controls have BF16-relative gaps of 0.020 nats at both model sizes, recorded to three decimal places.

D.2 What do the paired 181M studies establish?

The 181M model has 181,795,584 parameters, twenty blocks, width 768, twelve heads of dimension 64, and a global batch of 196,608 tokens. The nominal 100M-token configuration schedules 509 steps, or 100,073,472 tokens. Logging occurs every ten steps through step 500. For each paired seed, the reported training-loss gap averages the 21 common logged steps from 300 through 500. The across-seed statistic is the mean and sample standard deviation of those three within-seed averages.

Table 15 separates the projection gap from the increment obtained by adding FP4 attention to those projections. At seed 42, the increment is $0.0303 - 0.0176 = 0.0127$. Each column is a paired contrast. Their standard deviations are computed separately across seeds.

Table 15: **Paired 181M training-loss decomposition.** Mean gap over logged steps 300–500, in nats. The attention increment uses the same-seed projection-only run as its baseline.

Seed	Projection-only gap	Attention increment	Complete gap
42	0.0176	0.0127	0.0303
43	0.0119	0.0204	0.0323
44	0.0146	0.0173	0.0320
Mean ± SD	0.0147 ± 0.0029	0.0168 ± 0.0039	0.0315 ± 0.0011

A separate attention-only pair uses QK normalization, one shared seed, and identical data order for 2,600 steps, totaling 511,180,800 tokens. Its 200-step mean gap peaks at 0.067 nats in steps 800–999, reaches 0.034 in steps 1400–1799, and ends at 0.042 in steps 2400–2599. The two final-window gradient norms are 0.23. Final-step losses are 3.648 and 3.693, giving a 0.045-nat gap. Figure 2 retains the windowed trajectory.

D.3 What changes when the checkpoint is served in FP4?

The checkpoint study crosses training precision with serving precision. Fixed-batch validation uses 16 sequences of length 2,048 at 8B and 32 sequences at 1.7B. The same batches are used across serving modes within each scale. Direct casting evaluates BF16-trained weights in the FP4 path without evaluation-time calibration, outlier smoothing, or per-layer precision selection. The complete FP4-trained recipe includes smoothing during training.

For loss $\ell_{t,e}$ with training precision t and evaluation precision e, the checkpoint comparison is

$$\ell_{4,4} - \ell_{16,16} = (\ell_{4,16} - \ell_{16,16}) + (\ell_{4,4} - \ell_{4,16}). \tag{37}$$

At 8B, this is $0.0570 = 0.0222 + 0.0348$ nats; at 1.7B, it is $0.1154 = 0.0610 + 0.0544$. The first term compares trained checkpoints under BF16 serving. The second measures the additional FP4-serving cost on the FP4-trained checkpoint. Table 4 gives the complete validation grid.

Tasks and metrics. The seven-task suite is ARC-Easy, ARC-Challenge, HellaSwag, PIQA, WinoGrande, BoolQ, and LAMBADA (Clark et al., 2018; Zellers et al., 2019; Bisk et al., 2020; Sakaguchi et al., 2020; Clark et al., 2019; Paperno et al., 2016). ARC, HellaSwag, and PIQA use normalized accuracy; WinoGrande, BoolQ, and LAMBADA use accuracy. LAMBADA uses the `lambada_openai` variant. The average is the unweighted mean across tasks. The 8B entries are full-set point estimates. The 1.7B table retains the per-task evaluation uncertainties.

Under BF16 serving, the 8B FP4-minus-BF16 training deltas range from -0.8 to $+0.8$ points. They are $+0.8$ on ARC-Easy, -0.8 on ARC-Challenge, $+0.3$ on PIQA, -0.4 on WinoGrande, $+0.1$ on BoolQ, -0.1 on LAMBADA, and -0.4 on HellaSwag. The mean changes from 75.0% to 74.9%. Both training arms lower BoolQ accuracy relative to the original checkpoint, which scores 86.6%.

Under FP4 serving, the 1.7B FP4-trained checkpoint scores 62.3%, compared with 60.4% for direct casting. Under BF16 serving, its mean is 65.0%, compared with 66.0% after BF16 training. These four cells exhibit the same serving-dependent ordering as the validation grid.

Table 16: **Qwen3-8B downstream evaluation after 200M tokens.** Percentages. The FP4-trained checkpoint uses native FP4 products during training. Base denotes the original checkpoint.

Training Serving	Base BF16	BF16 BF16	FP4 BF16	FP4 FP4
ARC-Easy	80.9	82.5	83.3	82.2
ARC-Challenge	56.7	59.2	58.4	56.6
PIQA	77.5	78.9	79.2	78.4
WinoGrande	67.7	73.3	72.9	70.2
BoolQ	86.6	84.7	84.8	83.7
LAMBADA	64.2	70.2	70.1	68.0
HellaSwag	74.9	76.0	75.6	74.2
Average	72.7	75.0	74.9	73.3

Table 17: **Qwen3-1.7B downstream evaluation after 600M tokens.** Percentages with the per-task evaluation uncertainties. FP4 serving of BF16-trained weights is direct casting.

Training Serving	BF16 BF16	BF16 FP4, direct cast	FP4 BF16	FP4 FP4
ARC-Easy	74.0 ± 0.9	68.4 ± 1.0	74.5 ± 0.9	71.4 ± 0.9
ARC-Challenge	47.4 ± 1.5	43.6 ± 1.4	48.2 ± 1.5	42.6 ± 1.4
HellaSwag	63.8 ± 0.5	59.4 ± 0.5	62.3 ± 0.5	59.8 ± 0.5
PIQA	74.4 ± 1.0	70.0 ± 1.1	73.7 ± 1.0	71.8 ± 1.0
WinoGrande	64.2 ± 1.3	57.5 ± 1.4	63.2 ± 1.4	61.3 ± 1.4
BoolQ	78.4 ± 0.7	72.4 ± 0.8	76.3 ± 0.7	74.8 ± 0.8
LAMBADA	60.0 ± 0.7	51.3 ± 0.7	56.8 ± 0.7	54.4 ± 0.7
Average	66.0	60.4	65.0	62.3

D.4 Validation checks and reproducibility

The implementation suite contains 110 CPU tests and 72 passing native FP4 component cases, covering rounding, attention, replay, and native/reference products. The accompanying code includes deterministic CPU checks for the three STE placements, value-gradient normalization, covering scales, product moments, saved-factor recast bias, and propagated/fresh error separation. These checks test the algebraic identities underlying the method.

E Systems Configurations and Storage Components

E.1 B200 storage policy

The Qwen3-8B system has 8.19B parameters and uses eight B200 GPUs, tensor parallelism eight, sequence parallelism, and micro-batch two. Each GPU has a software-visible 167.6 GiB budget. The storage comparison recomputes SwiGLU intermediates. Its BF16 baseline retains Q, K, V, O with K, V expanded to all 32 query heads. Other saved activations include one tensor per RMSNorm. The QRO column includes packed attention state and query-row ownership. Additional packing covers working weights and non-attention factors.

The FP32 master and two Adam moments total $3.81 + 7.63 = 11.44$ GiB per GPU. BF16 working weights and gradients add 1.91 GiB each. Cross-entropy uses 8,192-token chunks. Framework buffers contribute 4.50 GiB. The table separates fragmentation allowance from stored tensors and buffers.

Head layout and packing. For the same retained elements, NVFP4 payloads and microscale bytes use $9/32$ of BF16 storage. Head layout independently changes the number of retained elements. For Q, K, V, O, a 32-query/8-key-value-head layout contains

$$\frac{32 + 8 + 8 + 32}{32 + 32 + 32 + 32} = \frac{5}{8} \tag{38}$$

Table 18: **B200 memory components at 2,048 tokens.** GiB per GPU. The BF16 column uses expanded K/V. Parameter rows are byte counts; the remaining rows follow the reported storage accounting.

Component	BF16, expanded K/V	QRO	QRO + packed
FP32 master weights	3.81	3.81	3.81
FP32 Adam moments	7.63	7.63	7.63
Working weights	1.91	1.91	0.54
Gradient storage	1.91	1.91	1.91
Attention-state subtotal	0.56	0.13	0.13
Other saved activations	0.28	0.28	0.08
Activation gradients, one layer	0.06	0.06	0.06
FP32 partial-dQ buffer	0.01	0.00	0.00
Chunked cross-entropy logits	0.58	0.58	0.58
Framework buffers	4.50	4.50	4.50
Fragmentation allowance	0.15	0.10	0.08
Total	21.40	20.91	19.31

of the elements in the expanded 32-head layout, before metadata and temporary storage. Query-row ownership changes accumulator lifetime. These are separate storage choices. The attention-state subtotal is separate from working weights, parameter gradients, other saved activations, and the partial-dQ buffer.

Auxiliary-output lifetime. An auxiliary output retained across all saved layers adds one tensor per layer with $BTH_q d$ values before sharding. Recomputing it replaces persistent storage with a live output tile, accumulator, and a tiled PV_f product. Once $D_\star$ has been reduced, the corresponding auxiliary rows can be released. Complete-step timings include the auxiliary product.

E.2 Additional B200 contexts

Table 19 gives the remaining short-context rows. Section 5.4 contains the longer-context comparison. At 2,048 tokens, the central complete-step rate is 235k tokens/s for FP4 and 151k for BF16. At 8,192 tokens, the rates are 224k and 139k. Speedups use the recorded paired ratios.

Table 19: **Additional Qwen3-8B B200 settings.** Step rates are k tokens/s and memory is GiB per GPU. Both precisions use SwiGLU recomputation. The BF16 storage column expands K/V.

	Step rate			Memory/GPU		
Context	BF16	FP4	Speedup	BF16	QRO	+ packed
2,048	151	235	1.56×	21.4	20.9	19.3
4,096	–	–	–	22.4	21.4	19.6
8,192	139	224	1.61×	24.4	22.4	20.2

The full-checkpoint BF16 memory sweep is 20.7, 21.0, 21.6, 25.1, 39.1, 57.8, and 95.2 GiB at contexts 2,048, 4,096, 8,192, 32,768, 131,072, 262,144, and 524,288, respectively. Full checkpointing is enabled in the longest BF16 timing row. The remaining complete-step timing rows use SwiGLU recomputation.

E.3 Eight-RTX accumulation-schedule study

The eight-RTX PRO 6000 system shards parameters, gradients, and Adam moments by tensor parallelism. Its micro-batch is one, residual stream is BF16, and activation checkpointing is disabled. Table 20 separates the reported attention state from optimizer state. Main-text Table 7 contains the 13B/32B total-memory comparison. The 7B totals are given here alongside their attention components.

Table 20: **Attention-state components on eight RTX GPUs.** GiB per GPU. "Global FP4" is the baseline FP4 accumulation schedule; QRO uses query-row ownership.

			Attention state		
Model	Context	Optimizer	BF16	Global FP4	QRO
7B	32,768	13.0	4.0	3.9	0.9
7B	65,536	13.0	8.0	7.8	1.8
7B	262,144	13.0	32.1	31.0	7.0
13B	131,072	24.2	25.1	24.2	5.5
13B	262,144	24.2	50.2	48.4	10.9
32B	65,536	59.6	32.1	31.0	7.0
32B	131,072	59.6	64.2	62.0	14.0

At 7B, total BF16/QRO memory is 19.1/15.9 GiB at 32,768 tokens, 25.1/18.8 at 65,536, and 61.2/36.0 at 262,144. At 65,536 tokens, the attention-state entries are 8.0 GiB for BF16, 7.8 for global-buffer FP4, and 1.8 for QRO. The comparison illustrates the importance of the accumulation schedule alongside packed operands.

Fully materialized intermediates. For batch B, heads H, and lengths T_q, T_k, one packed intermediate costs approximately $9BHT_qT_k/16$ bytes for payloads and scales. Materializing both P_f and dS therefore costs approximately $9BHT_qT_k/8$ bytes. At $B = H = 1$ and $T_q = T_k = 8,192$, this is 72 MiB. Tiled execution regenerates these intermediates and accumulates gradients without these complete arrays.